

## 4. General Design and Architecture Choices

### 4.1 - Design Requirements and Principles

The architecture follows the requirements in section 1.5. It must make source charges traceable, validate data before publication and serve consistent indicators to users with different access rights. The distinction between these requirements and the available tools matters because the local POC, cloud prototype and possible enterprise deployment have different operating conditions.

#### Functional drivers

- ingest recurring FOCUS cost and usage data;
- preserve received evidence and its provenance;
- validate a versioned data contract before canonical publication;
- publish consistent FinOps data products for Inform, Optimize and Operate;
- support dashboard users and authorized analytical consumers;
- reconcile row counts and financial measures across transformations.

#### Quality and governance drivers

- explicit ownership and definitions;
- controlled schema evolution;
- record-level and batch-level lineage;
- reproducible processing and validation;
- visibility of rejected, missing or unallocated information;
- separation between canonical data and derived analytical products.

#### Security drivers

- least-privilege and read-only analytical access;
- isolation of managerial or domain perimeters;
- protection of direct SQL access in addition to dashboard filtering;
- no credentials committed to source control.

#### Operational drivers

- restart and recovery information;
- observable freshness and execution status;

- maintainability by the platform and FinOps roles;
- portability between the local validation environment and the target cloud environment where feasible.

### **Project constraints**

The design must remain achievable within the PFE schedule and available licences. Confidentiality limits direct use of enterprise billing data. Only restricted anonymized reference files are available. The local DuckDB POC and the DEV and PROD catalogs of the Databricks/GCP prototype provide different levels of evidence. Deployment into an enterprise environment remains outside the implemented scope.

## **4.2 - General Platform Design (Logical HLD)**

### **System boundary**

The system begins with cost and usage datasets delivered by a billing-data producer and ends with analytical products for FinOps. Perimeter-restricted access for managers and analysts is a design requirement, not an implemented capability established by the current dashboard. Provider billing generation, enterprise accounting, resource remediation and production identity administration remain external responsibilities.

### **Logical flow**

```
Cost and usage sources
    v
Source retention and lineage
    v
Ingestion and lineage
    v
Contract and quality validation
    v
Canonical cost and usage data
    v
Dimensional and subject-oriented data products
    v
Dashboard and analytical access (perimeter controls: target)
```

Quality, reconciliation, security, metadata and observability apply across the complete flow.

## Logical components

| Component            | Responsibility                                                                                                                                         |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source interface     | Receive periodic cost and usage data with identifiable origin and period                                                                               |
| Evidence store       | Retain the received representation and its provenance without hidden business transformation; source immutability requires separate enforcement        |
| Ingestion service    | Register the batch and add technical lineage                                                                                                           |
| Contract gate        | Reject incompatible structure and invalid mandatory values before canonical publication; the current prototype does not implement a quarantine dataset |
| Canonical repository | Provide the authoritative standardized FinOps representation                                                                                           |
| Analytical model     | Organize detailed charges by time, scope, resource, service, price and commitment                                                                      |
| Data-product layer   | Publish stable subjects and common measures for consumption                                                                                            |
| Consumption layer    | Provide role-appropriate dashboards and analytical access                                                                                              |
| Operational control  | Track executions, freshness, failures, reconciliation and recovery                                                                                     |

## Architectural principles

- Retained source evidence and the canonical repository have different responsibilities; retention alone does not prove immutability.
- Validation occurs before data becomes authoritative.
- Derived products do not become independent sources of truth.
- Consumers use stable products rather than storage-specific implementation details.
- Access control applies at every exposed interface.
- Claims of reliability require reproducible evidence, not only an architecture diagram.

## Operating model: centralized production and distributed consumption

The proposed operating model gives managers access to their cost scope and authorized analysts access to further analysis. The Data Platform retains shared technical capabilities; FinOps owns business definitions and product validation; domains consume only their authorized scope. These responsibilities describe the target design and require organizational confirmation.

Here, “data factory” denotes a centralized production organization, not the Azure Data Factory service. Centralized production can coexist with autonomous access. The issue under study is dependency on repeated requests for changes.

Data Mesh combines domain ownership, data as a product, self-service infrastructure and federated governance. Its autonomy extends to producing teams, beyond report consumption. Physical data location alone does not define it (Dehghani, 2020).

The target adopts product-oriented practices-definitions, quality, ownership and stable interfaces-without autonomous production by several domains or full federated governance. It is therefore not described as a complete Data Mesh. Chapter 9 will revisit this choice against the evidence collected.

This section describes what the platform must do without assigning those responsibilities to products. Section 4.3 compares the documented technology options and identifies the tools used in the prototypes. Section 4.4 maps the logical flow to deployed components; section 4.5 explains the data model and the analytical products.

Source for this section (accessed 30 September 2026):

- Dehghani, *Data Mesh Principles and Logical Architecture* (2020):  
<https://martinfowler.com/articles/data-mesh-principles.html>

## 4.3 - Technology Options and Implemented Choices

### 4.3.1 Evaluation basis

The project's initial architecture notes considered Microsoft Fabric, Azure Databricks and a modular Azure PaaS solution against requirements for ingestion, contract validation, governed analytical products and reporting. They also listed maintainability, security, cost, available licences and feasibility within the PFE as criteria. This was qualitative exploration, not a completed selection assessment: no final comparative decision criteria, weighted scores or measured total costs for the three candidates were recorded. The notes identify risks but cannot support a numerical ranking.

### 4.3.2 Platform alternatives and implemented scope

Fabric groups data preparation, storage, SQL and Power BI within one service environment. The initial record treats this integration as a possible advantage and flags capacity and licensing as questions. Azure Databricks offers a lakehouse processing path with Unity Catalog and SQL serving, but the proposed Azure architecture would also require storage, identity and reporting integration. A modular Azure solution would make service boundaries explicit while increasing the number of components to configure and operate. These are design observations from the initial evaluation, not benchmark results.

The cloud prototype uses Databricks with GCP object storage. Its code and catalog organization are described in sections 4.4 and 5.3; execution evidence is assessed in section 7.4. This is an implemented choice, not proof that Databricks/GCP ranked first in the initial comparison. The available decision record does not isolate the commercial, access or scheduling reason for the move from the Azure-first options. The thesis therefore does not present the GCP prototype as an approved enterprise deployment or infer that it outperformed Fabric on cost.

### 4.3.3 Storage and processing choices

The independent generator publishes Parquet files as the shared RAW input. Apache Parquet specifies a column-oriented file format for storage and retrieval (Apache Parquet, n.d.). The Databricks pipeline publishes managed Delta tables in Bronze, Silver and Gold, where month-level replacement and table history are required. Delta Lake extends Parquet data files with a transaction log (Databricks, n.d.-e). The two formats therefore serve different roles in this design rather than being mutually exclusive alternatives. A replacement is atomic for an individual Delta table operation, not for the entire sequence of Silver, Gold and datamart writes. A failure between these operations requires audit and replay.

Python and PyArrow support deterministic file generation and schema checks. SQL defines the cloud Gold model and datamarts, and PySpark coordinates their execution in Databricks. YAML records the versioned data contract. DBML documents the earlier POC model; the cloud SQL DDL, not that DBML file, is the source of truth for current table definitions. DuckDB serves the frozen local POC; its results are evidence for that environment only.

### 4.3.4 Reporting decision and remaining comparison

The POC uses Streamlit with DuckDB datamarts, whereas the cloud application is designed to query Databricks through a SQL Warehouse. The existing Power BI dashboard remains the professional reference system. The repository demonstrates local Streamlit execution, but it does not contain a controlled comparison of these reporting options or validated Row-Level Security for the new application. The selection can therefore be defended for the implemented prototype and its reproducible tests; an enterprise adoption recommendation would also require access, performance and operating-cost evidence.

### 4.3.5 Technology choices by function

The following table summarizes the tools present in the two implementations. The local POC is a separate experiment, not the DEV environment of the cloud platform. DEV and PROD use the same Databricks/GCP architecture with separate business catalogs. The table reports implementation choices and their role; it is not a scored comparison of every tool named during early exploration.

| Function             | Local POC                                            | Cloud DEV and PROD                                                                              | Rationale and evidence boundary                                                                                                                                                                          |
|----------------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source files         | Synthetic Parquet files on the local filesystem      | A manually published copy in GCS, exposed through an external Unity Catalog volume              | The intended common input supports comparison only when publication and cloud ingestion are checked separately.                                                                                          |
| Data processing      | Python/Pandas and local notebooks                    | PySpark for ingestion and contract processing; versioned SQL for Gold and datamarts             | The cloud path uses distributed Spark execution, but no performance advantage follows without a comparable benchmark.                                                                                    |
| Processed tables     | Local Delta layers and a DuckDB analytical warehouse | Managed Delta tables in Bronze, Silver, Gold and datamarts                                      | Delta supports table-level replacement and history; DuckDB supplies local analytical tests.                                                                                                              |
| Orchestration        | Explicit local commands and notebook execution       | Databricks Jobs coordinating notebooks and validation tasks                                     | Historical backfill Jobs exist; a complete Daily DEV-to-PROD run and Classic monthly-close Job still require separate verification. An automatic schedule does not by itself prove end-to-end operation. |
| Governance and audit | Local contract checks and test outputs               | Unity Catalog namespaces and permissions; <code>finops_ops.audit</code> for operational history | Catalog organization is implemented, while effective access and recovery still require run-level tests.                                                                                                  |
| Analytical access    | DuckDB queries over local datamarts                  | SQL Warehouse queries over PROD datamarts and OPS audit                                         | The Warehouse serves the application; it is not the compute engine used by every batch task.                                                                                                             |
| Visualization        | Local Streamlit application                          | Separate Streamlit application packaged for Cloud Run                                           | The local interface is tested; Cloud Run page behavior and permissions need their own evidence. Power BI remains the existing professional reference system.                                             |

Sources for this section (accessed 30 September 2026):

- Apache Parquet, *Overview*: <https://parquet.apache.org/docs/overview/>
- Databricks, *What is Delta Lake in Databricks?*: <https://docs.databricks.com/gcp/en/delta/>

## 4.4 - Selected Physical Architecture

Section 4.2 defines the required functions, and section 4.3 explains the chosen technologies. This section places the implemented components in their actual storage, catalog, processing and access boundaries. It answers where data resides and how the components connect; it does not repeat the technology comparison or detail every table, which is the purpose of section 4.5.

### 4.4.1 System boundary and implementation tracks

The implementation separates three responsibilities. `FinOps Data Generator` produces a single FOCUS-based synthetic dataset, independently of any analytical engine. `FinOps Data Platform - POC` preserves the earlier DuckDB/Streamlit experiment and its January 2025 local results. `FinOps Cloud Data Platform` is the active Databricks/GCP implementation. This division preserves the value of the local validation without implying that DuckDB and Spark execute identical physical code or that local results validate cloud operation.

The cloud platform receives detailed Daily and monthly Parquet files in a shared RAW namespace, transforms them through Bronze and Silver Delta tables, builds a Gold dimensional model, and publishes subject-oriented datamarts. The repository packages the final Streamlit dashboard as a container for Google Cloud Run. The application queries PROD datamarts and operational audit through a SQL Warehouse using a read-only technical identity; its deployed behavior and effective permissions still require retained verification evidence. Read-only access does not, by itself, limit each viewer to an application or domain. The architecture is a PFE prototype with planned extensions; it is not a claim of deployment into Technip Energies production systems.

```
Independent generator -> GCS source bucket -> finops_raw.landing.focus
                                     |
                                     | finops_dev -> Bronze -> Silver -> Gold -> datamarts
                                     |
                                     | finops_prod -> Bronze -> Silver -> Gold -> datamarts
managed Delta tables and OPS audit -> separate UC storage bucket
DEV and PROD runs -> finops_ops.audit
PROD datamarts + audit -> SQL Warehouse -> Cloud Run/Streamlit
```

### 4.4.2 Source authority, catalogs and data lifecycle

`finops_raw.landing.focus` exposes Parquet objects from `gs://dtl_finops/focus` to both environments. DEV business tables reside in `finops_dev`; PROD business tables reside in `finops_prod`. Each contains Bronze, Silver, Gold and datamart schemas. The separate `finops_ops.audit` catalog has five operational tables and records an explicit environment on each row. In the Classic Compute setup used for the Belgium prototype, managed catalog storage is configured under `gs://dtl_finops-unitycatalog-euwl/catalogs/`, separate from the source bucket. This arrangement permits independent data publication while maintaining a shared operational history. It also makes an environment filter essential in every audit lookup and write.

The source lifecycle follows financial meaning. Daily files populate an open month provisionally. The detailed monthly billing file becomes authoritative after close: the pipeline replaces that month separately in the two Silver tables and in the Gold fact, merges Gold dimensions, and rebuilds datamarts. These writes are not one cross-table transaction. The two source types are never added together in the fact table; replay and audit are needed if a run stops between table operations.

Automatic RAW archival is disabled in the common configuration. The earlier design would have moved processed sources after close, but a DEV close could then remove a file needed for independent PROD processing. Keeping the RAW objects allows both environments to consume the same path for replays. It does not make those objects immutable: Bronze ingestion deduplicates on `_source_file`, so replacement of an object at the same path is not detected as a new source version by that check. Source-version or checksum controls would be needed to establish immutable evidence. Retention also has a storage cost; any future move or deletion must account for every consumer and the audit period. The archival module remains optional, not an active lifecycle step.

#### 4.4.3 Compute configuration boundary

The repository separates common processing logic from compute-specific deployment artifacts. `platform/common` contains shared notebooks and SQL. `platform/serverless` and `platform/classic_compute` contain the respective setup and Job adapters. The Belgium implementation uses the Classic setup with explicit GCS managed locations; the Serverless setup remains a distinct execution scenario using Default Storage. Both target the same logical RAW-to-datamart flow, but their catalog storage configuration, compute lifecycle and billing attribution differ. The separation avoids redefining the data contract or analytical grain when adapting the deployment.

This architecture section records the physical components and their interfaces, not the chronology of workspace decisions or a claim that either compute mode is more economical. Chapter 7 records execution evidence and chapter 8 examines the later location and compute choices as an operating-cost case study.

#### 4.4.4 Analytical and consumption layers

Silver retains the canonical FOCUS representation after contract validation. Gold and the fourteen datamarts are stored separately in each business catalog. Versioned SQL defines and refreshes these products; Python orchestrates execution and supplies validated environment identifiers. Some datamarts read central Silver fields absent from the Gold fact, while others read Gold. Section 4.5 specifies the tables, their grain and the business questions they are designed to address. Each datamart is fully refreshed in the current implementation; this simplifies publication but adds work after a monthly replacement and can restate historical groupings when Type 1 dimension attributes change.

The frozen local dashboard's query definitions reference ten of the fourteen local datamarts; this count describes code usage, not a separate execution result. The cloud dashboard is a separate Streamlit implementation packaged for Cloud Run and designed to read PROD datamarts and OPS audit in read-only mode. Its technical identity can restrict the application to serving objects, but it does not identify which viewer is a domain manager, application owner or project manager. No viewer-specific perimeter policy is implemented in the repository at the time of this design update. Access must therefore remain limited to synthetic demonstration data or trusted project users until the proposed authorization controls are implemented and tested. Effective grants and direct SQL access still require verification; interface behavior alone cannot establish them.

#### 4.4.5 Dashboard identity and authorization boundary

The proposed access-control extension separates authentication from business authorization. In the production target, Google Identity-Aware Proxy authenticates a viewer before a request reaches Cloud Run. The application must validate the signed IAP assertion rather than trust an email supplied by a form or an unsigned header. It then resolves the verified email against two managed control tables in a new `finops_ops.security` schema. `user_entitlement` associates a principal with a role and scope, while `business_scope` relates domains, subdomains, applications and projects to attributes available in the FinOps products. The application service principal remains read-only and receives no RAW, DEV or write privilege.

The PFE demonstration uses the same authorization logic with a fixed list of synthetic personas. A configuration value selects either `demo`, in which a predefined persona can be chosen, or `iap`, in which no selector is exposed and the signed identity is mandatory. A free-text email field is excluded because it would allow a viewer to assert another identity. The demonstration can therefore test role and scope behavior without being presented as enterprise authentication. The planned roles are FinOps administrator, domain manager, subdomain manager, application owner and project manager; an identity without an active entitlement must be denied by default.

*Source: Google Cloud, Configure IAP for Cloud Run and Getting the user's identity (accessed 1 October 2026).*

This boundary is an architectural decision and validation target, not yet an implemented result. Section 5.3 defines the intended implementation and Section 7.5 defines the evidence required before the thesis can claim successful role- and scope-based isolation.

#### 4.4.6 Architectural trade-offs and evidence boundary

The design prioritizes a shared source path, separate DEV and PROD catalogs, a canonical Silver layer and reproducible SQL products. It accepts duplicated business data, materialized datamarts and retained RAW objects to make testing, promotion and audit easier. It does not yet guarantee source immutability or user-specific showback access. The proposed persona simulation reduces implementation risk and demonstrates authorization logic, but only validated IAP integration and organizational identities could support a production authentication claim. These choices incur storage, refresh and operational costs that chapter 8 must quantify. Chapter 7 must establish which cloud paths ran successfully and how they compare with the POC. The existence of two compute configurations alone cannot establish a cost or latency advantage.

### 4.5 - Data Architecture and Dimensional Modeling

#### 4.5.1 Canonical record and analytical grain

The data architecture separates three purposes. RAW retains the received Parquet evidence; Bronze adds technical lineage without changing source business values; and Silver contains the FOCUS representation after the versioned contract has accepted it. Gold reorganizes that canonical record for recurring analysis, while datamarts answer defined questions at coarser grains. Neither Gold nor a datamart becomes an independent source of cost truth. The same logical model is instantiated in separate DEV and PROD catalogs; the frozen local POC uses a related analytical design but a different engine.

The Gold fact, `fact_finops_cost_usage`, is designed to represent one FOCUS cost or usage charge line. It retains `billing_month` so that an authoritative monthly bill can replace the provisional Daily-fed month without adding both sources together. Its technical key is derived

from the source path and a row number assigned within that file; dimensional foreign keys connect a charge to its time, scope, resource, service, SKU, location, pricing and charge type. The row number is ordered by selected business fields, but those fields may tie for distinct rows. The current implementation therefore does not guarantee a stable key across replays for such ties. The charge-line grain permits reconciliation back to Silver, but does not itself establish the financial meaning of a difference between cost columns.

#### 4.5.2 Gold table inventory and relationships

The cloud SQL defines the following Gold tables. This inventory gives each name and responsibility. A full column-by-column DDL listing is not required to interpret the model in this chapter.

| Table                                | Analytical responsibility                                            |
|--------------------------------------|----------------------------------------------------------------------|
| <code>fact_finops_cost_usage</code>  | Charge-line measures, quantities, billing month and dimensional keys |
| <code>dim_date</code>                | Reusable calendar attributes for the fact's dates                    |
| <code>dim_billing_scope</code>       | Billing accounts, subaccounts, customers and cost-center attributes  |
| <code>dim_resource</code>            | Resource, resource group and application/owner attributes            |
| <code>dim_service</code>             | Service, provider, publisher and reseller attributes                 |
| <code>dim_sku</code>                 | SKU, meter, offer and term attributes                                |
| <code>dim_location</code>            | Region and availability-zone attributes                              |
| <code>dim_pricing</code>             | Pricing category, unit and currency attributes                       |
| <code>dim_commitment_discount</code> | Commitment identifiers and categories                                |
| <code>dim_charge_type</code>         | Charge category, subcategory and frequency                           |
| <code>dim_tag</code>                 | Normalized tag key/value pairs                                       |
| <code>bridge_resource_tag</code>     | Many-to-many association between resources and tags                  |

The model has ten dimensions. The fact links directly to nine of them; tags are reached through `dim_resource`, `bridge_resource_tag` and `dim_tag`. The billing-scope and resource dimensions use Type 1 updates. Their validity columns do not preserve SCD2 history: a later cost-center or application-owner update may change how an earlier charge is grouped when the datamarts are refreshed. This is a consequential limitation for historical showback. The source dataset lacks a native charge identifier and availability zone. The generated charge identifier inherits the row-order limitation described in section 4.5.1, while the unavailable

zone is recorded as `Unknown`. These are implementation limitations, not general properties of FOCUS.

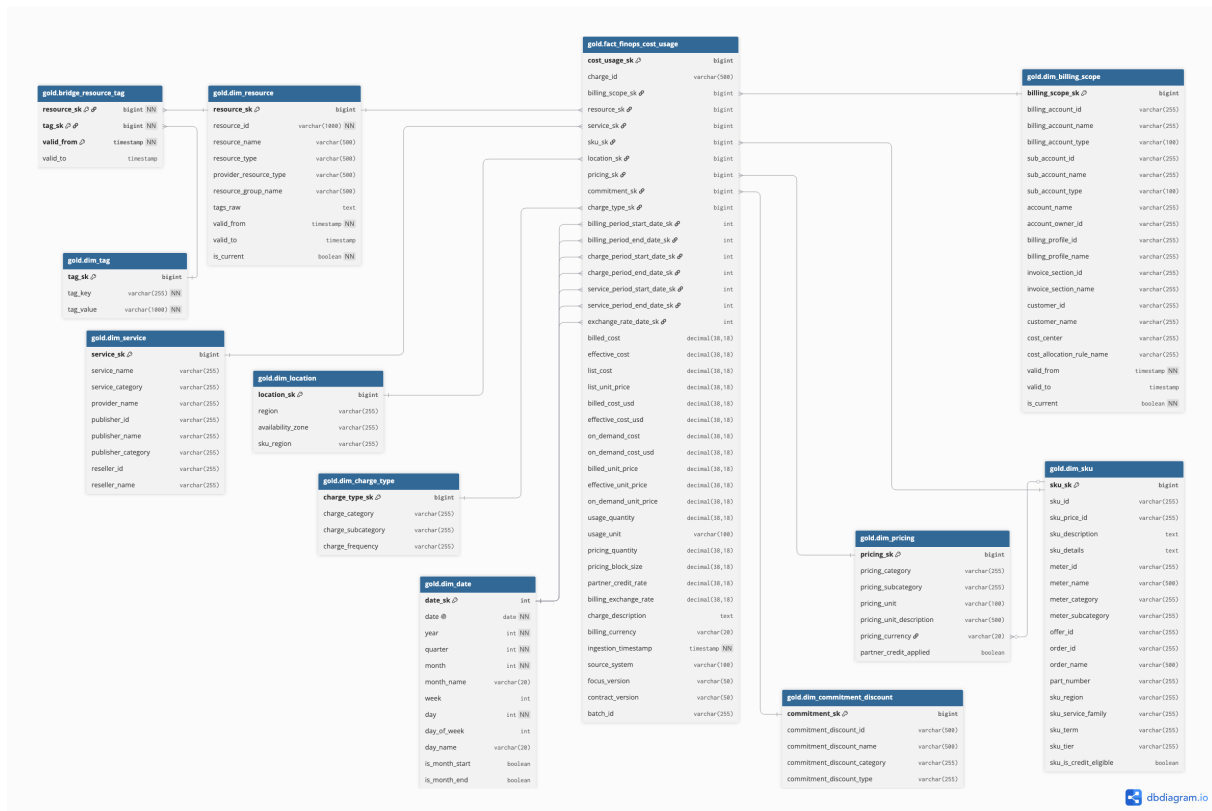


Figure 4.1 - Gold dimensional relationships in the local POC. The diagram illustrates the fact-to-dimension structure and the resource-tag bridge; its column types and surrogate-key definitions are not the physical Databricks implementation. In particular, the POC diagram uses `BIGINT` keys where the cloud Gold DDL defines mainly `STRING` keys. Source: project POC data model and cloud Gold DDL.

### 4.5.3 Datamarts and business questions

The datamarts are analytical products derived from the canonical representation. The question column states their intended use, not a measured benefit or a claim that every cloud table has been independently validated. The paragraph after the table distinguishes products derived from Gold from those reading central Silver directly.

| Datamart                       | Business or control question supported                                      |
|--------------------------------|-----------------------------------------------------------------------------|
| dm_monthly_billing             | What is the billed cost for each billing month?                             |
| dm_daily_billing               | How does billed cost vary by charge-start date?                             |
| dm_cost_by_scope_service_month | Which cost centers, customers and services account for monthly billed cost? |

| <b>Datamart</b>                                 | <b>Business or control question supported</b>                                          |
|-------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>dm_top_services</code>                    | Which service categories and names have the highest billed cost in the loaded data?    |
| <code>dm_top_resources</code>                   | Which resources have the highest billed cost in the loaded data?                       |
| <code>dm_cost_by_charge_type</code>             | How is monthly billed cost distributed across charge categories and frequencies?       |
| <code>dm_sku_cost</code>                        | Which SKUs and meters account for the highest billed cost?                             |
| <code>dm_savings_monthly</code>                 | How do monthly List, Contracted and Effective Cost totals differ?                      |
| <code>dm_executive_summary_monthly</code>       | What are the monthly cost totals, charge-line count, resource count and service count? |
| <code>dm_top_resources_monthly</code>           | Which resources drive billed cost within a selected billing month?                     |
| <code>dm_data_quality_monthly</code>            | How many critical nulls and ingestion batches appear in each Silver month?             |
| <code>dm_cost_by_resource_group_month</code>    | What are monthly billed cost and resource count by resource group?                     |
| <code>dm_cost_by_subscription_month</code>      | What are monthly billed cost and resource count by subaccount or subscription?         |
| <code>dm_cost_by_application_owner_month</code> | How are monthly billed cost and resource count attributed to applications and owners?  |

`dm_savings_monthly`, `dm_executive_summary_monthly` and `dm_data_quality_monthly` read central Silver because they use FOCUS fields or technical metadata not all held in the Gold fact; `ContractedCost`, for example, is absent from that fact. The remaining datamarts read Gold. The quality datamart describes rows that reached Silver; it does not count inputs blocked by the contract. Columns named “savings” in the SQL contain arithmetic differences between cost measures; they are not validated realized savings without eligibility rules, charge-category analysis and business review. Section 5.3.3 describes how the cloud model and datamarts are built, while chapters 6 and 7 examine their analytical meaning and validation evidence.

## References

Apache Parquet. (n.d.). *Overview*. <https://parquet.apache.org/docs/overview/> (accessed 30 September 2026).

Databricks. (n.d.-e). *What is Delta Lake in Databricks?* <https://docs.databricks.com/gcp/en/delta/> (accessed 30 September 2026).

Dehghani, Z. (2020, 3 December). *Data Mesh Principles and Logical Architecture*. MartinFowler.com. <https://martinfowler.com/articles/data-mesh-principles.html> (accessed 30 September 2026).

Google Cloud. (n.d.-a). *Configure IAP for Cloud Run*. <https://cloud.google.com/run/docs/securing/identity-aware-proxy-cloud-run> (accessed 1 October 2026).

Google Cloud. (n.d.-b). *Getting the user's identity*. <https://cloud.google.com/iap/docs/identity-howto> (accessed 1 October 2026).